Comparación de algoritmos

¿Qué algoritmo predice mejor la pobreza de los hogares?

Equipo 03: Julián Herrera, Andrés Silva y Valentina Vera

28 de octubre de 2026

MECA 4107 Big Data y Machine Learning para Economía Aplicada, Universidad de los Andes

Mejor modelo: LightGBM con 1.296 variables, 0,77 en Kaggle

Especificación

- Boosting de gradiente (LightGBM)
- Conjunto X_7: 1.296 variables del hogar y de sus miembros
- 31 hojas por árbol, tasa de aprendizaje 0,04, 748 rondas
- Penalidades L2 = 10 y L1 = 0,1
- Umbral 0,39, el que maximiza el F1 en validación cruzada

0,77

F1 en Kaggle (tabla pública)

F1 CV 0,768 · precisión 0,744 · recall 0,794

Lo que muestra la comparación

1. **El algoritmo importa:** con las mismas 43 variables, el boosting obtiene el mayor F1 (0,731 frente a 0,715 de Random Forest y 0,711 de Elastic Net).
2. **Gana porque ordena mejor a los hogares:** para encontrar a los mismos pobres, incluye unos 2.000 hogares no pobres menos que Elastic Net.
3. **La información importa más:** describir mejor el hogar suma 3,7 puntos de F1; ajustar hiperparámetros o rebalancear las clases, a lo sumo 1,1.

Datos: predecir la pobreza sin observar el ingreso

	Train	Test
Hogares	164.960	66.168
Personas	543.109	219.644
Variables de hogares	23	16
Variables de personas	135	63
Hogares pobres	20,0%	A predecir

Fuente: DANE, Misión de Empalme (MESE). Hogares y personas se unen con el identificador del hogar.

- Un hogar es pobre si su ingreso per cápita está por debajo de la línea de pobreza.
- En test no está el ingreso: faltan 7 variables de hogares y 72 de personas, casi todas montos de ingresos. Hay que predecir la pobreza con lo que se asocia al ingreso.
- Solo 1 de cada 5 hogares es pobre. Clasificar a todos como no pobres acierta el 80%, pero no encuentra a ningún pobre: su F1 es 0.
- Por eso evaluamos con F1 y elegimos el umbral que lo maximiza.

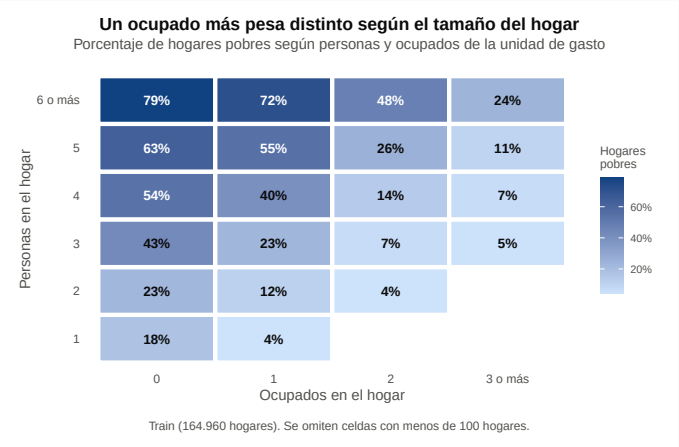
Predictores: de lo más fácil de observar a lo más detallado

Como no vemos el ingreso, usamos variables que lo aproximan: cuántos generan ingreso, cuánto y con qué calidad trabajan, entre cuántos se reparte y cuánto vale la vivienda.

Conjunto	Contenido	Variables	Algoritmos evaluados
X_1	Ubicación, tenencia de la vivienda, tamaño y hacinamiento	5	CART, Logit
X_2	+ composición del hogar, dependencia y educación	11	CART, Logit
X_3	+ ocupados e inactivos	15	CART, Logit
X_4	+ horas, cotización, pensión y posición ocupacional	21	CART, Logit
X_5	+ arriendo, desempleo, salud, subempleo y transferencias	43	Los 9 algoritmos
X_6	+ miembros por categoría de educación, ocupación y empresa	148	LightGBM, XGBoost
X_7	Detalle del hogar y de cada miembro (con datos faltantes)	1.296	LightGBM, XGBoost

Cada conjunto contiene al anterior y agrega un bloque; X_7 se construyó aparte y solo lo usan los algoritmos que manejan datos faltantes.

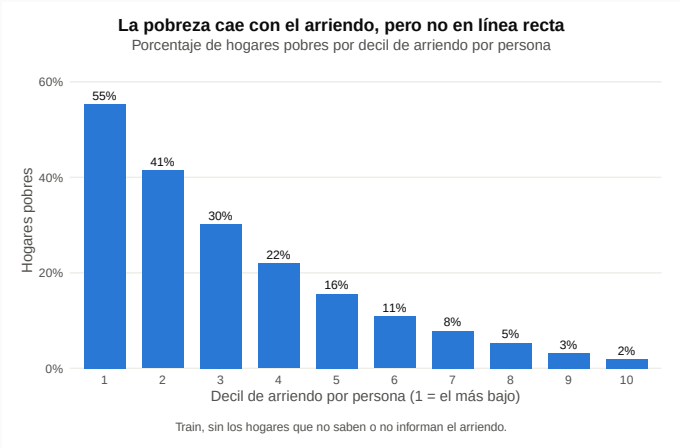
Datos: el efecto de un ocupado depende del tamaño del hogar



- Ocupados y personas están arriba y abajo en el ingreso per cápita: su efecto se combina.
- Pasar de 1 a 2 ocupados reduce la pobreza en 8 puntos en hogares de 2 personas y en 24 puntos en hogares de 6 o más.
- Un modelo lineal suma el mismo efecto en todas las celdas; un árbol puede aprender que el efecto cambia.

Hipótesis: si el efecto de una variable depende de otra, a un modelo de árboles le debería ir mejor que a uno lineal.

Datos: el arriendo aproxima el ingreso, pero no en línea recta



- El arriendo aproxima el valor de la vivienda, que refleja el ingreso.
- La pobreza cae del 55% al 2%, pero no pareja: 14 puntos del decil 1 al 2 y solo 1 del 9 al 10.
- Un modelo lineal necesita transformaciones hechas a mano; un árbol parte la variable donde cambia la pobreza.

Hipótesis: si el efecto cambia por tramos, a un modelo de árboles le debería ir mejor que a uno lineal.

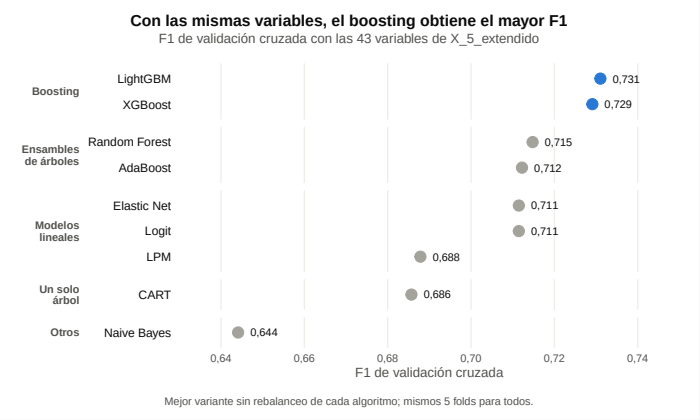
Probamos los siete algoritmos requeridos

Algoritmo	Variantes evaluadas	Mejor F1 CV	Kaggle (misma variante)
LPM	2	0,688	–
Logit	5	0,711	0,71
Elastic Net	1	0,711	–
CART	7	0,686	0,67
Random Forest	3	0,715	0,71
Boosting	8	0,768	0,77
Naive Bayes	2	0,644	0,64

- 28 variantes, todas evaluadas con los mismos 5 folds de validación cruzada.
- El F1 de validación cruzada anticipa bien el de Kaggle: la diferencia es de 0,016 o menos.
- Boosting incluye XGBoost, LightGBM y AdaBoost.
- El mejor resultado de cada algoritmo usa conjuntos distintos: para comparar algoritmos hay que fijar las variables.

–: esa variante no se envió a Kaggle. LPM y Elastic Net sí se enviaron, con otras especificaciones (0,68 y 0,69).

En igualdad de condiciones: las mismas 43 variables



- El boosting (0,731 y 0,729) queda separado del resto.
- **Por qué:** capta umbrales e interacciones sin que haya que especificarlas.
- Los ensambles y los modelos lineales quedan casi empatados (0,711 a 0,715).
- Un solo árbol (0,686) es inestable; Naive Bayes (0,644) supone variables independientes, y aquí no lo son.

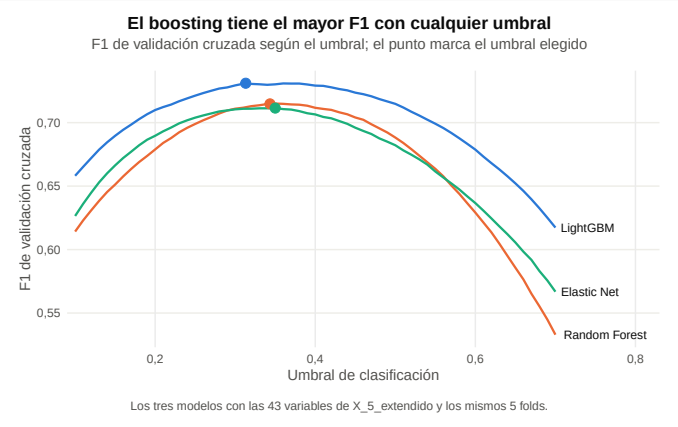
A fondo: el boosting reduce el undercoverage

Modelo	Umbral	Precisión	Recall	F1	Falsos positivos	Falsos negativos
LightGBM	0,31	0,659	0,820	0,731	13.996	5.936
Random Forest	0,34	0,651	0,792	0,715	13.985	6.880
Elastic Net	0,35	0,670	0,758	0,711	12.316	7.989

- Falsos negativos = **undercoverage**: hogares pobres que quedarían por fuera. Falsos positivos = **leakage**: hogares no pobres que recibirían el beneficio.
- Frente a Elastic Net, LightGBM encuentra 2.053 hogares pobres más, a cambio de 1.680 de leakage más.
- Frente a Random Forest, encuentra 944 hogares pobres más con el mismo leakage.
- Rivales: los dos algoritmos requeridos más cercanos al boosting con X_5 (Random Forest y Elastic Net), con los mismos folds.

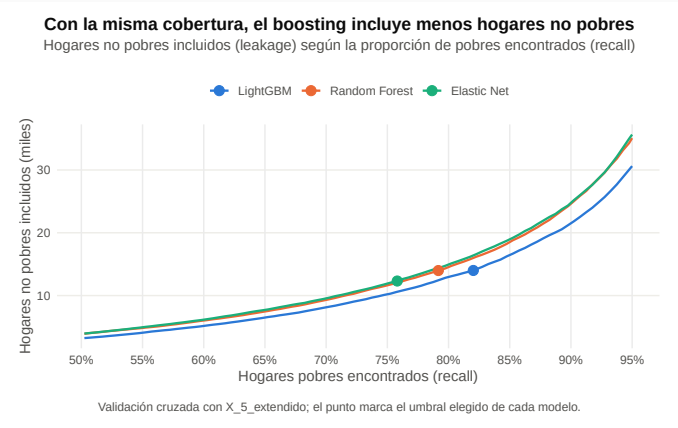
Validación cruzada en train (164.960 hogares). Cada modelo con el umbral que maximiza su F1.

A fondo: la ventaja no depende del umbral elegido



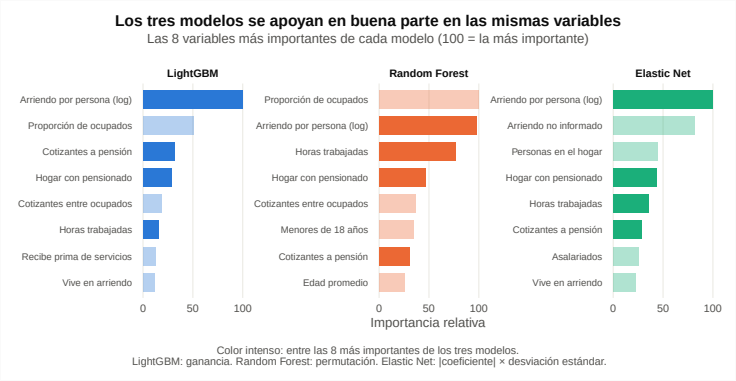
- LightGBM tiene el mayor F1 con cualquier umbral entre 0,10 y 0,70.
- Su ventaja no viene de haber escogido un buen umbral.

A fondo: a igual cobertura, menos leakage



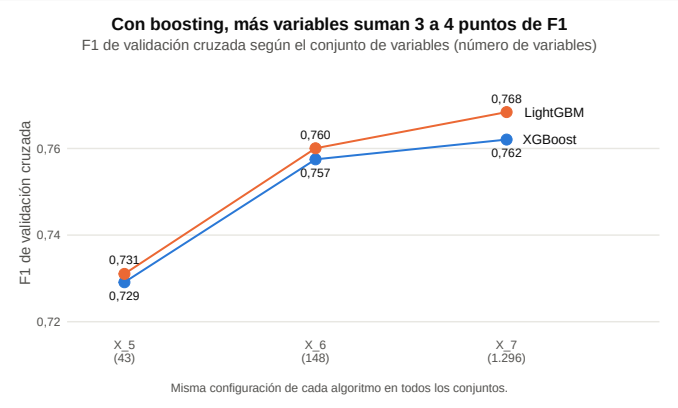
- Para encontrar al 80% de los hogares pobres, LightGBM incluye 13.088 no pobres; Random Forest, 14.743; Elastic Net, 15.135.
- La curva de LightGBM queda por debajo en todo el rango: ordena mejor a los hogares de más a menos probable de ser pobre.
- Por eso gana con cualquier balance entre leakage y undercoverage que prefiera la política.

A fondo: la misma información, con otro peso



- Cuatro variables están entre las 8 más importantes de los tres modelos: arriendo por persona, hogar con pensionado, horas trabajadas y cotizantes a pensión.
- La diferencia está en cómo las usan: Elastic Net suma efectos lineales; los árboles las combinan con umbrales, y el boosting además corrige en cada ronda los errores de la anterior.

Más información sobre el hogar pesa más que el algoritmo



- Con el mismo algoritmo, pasar de X_5 a X_6 (educación, ocupación y empresa de cada miembro) suma unos 3 puntos.
- X_7 agrega menos de 1 punto más con 1.148 variables adicionales: rendimientos decrecientes.
- Con X_7, LightGBM supera a XGBoost por 0,6 puntos.

Ajustar y rebalancear mueve poco el F1

Ajuste de hiperparámetros (X_5)

Variante	Hiperparámetros	F1 CV
Elastic Net	alpha 0,25; lambda 0,0001 (elegidos por CV)	0,711
Logit	Sin penalización	0,711
Random Forest (fijo)	mtry 6, nodo mínimo 10, 300 árboles (fijos)	0,704
Random Forest (grilla CV)	mtry 31, nodo mínimo 20, 300 árboles (elegidos por CV)	0,715

- Elegir los hiperparámetros de Random Forest por validación cruzada suma 1,1 puntos.
- Penalizar el logit (Elastic Net) no cambia el F1.

Rebalanceo de clases (X_5)

Algoritmo	Rebalanceo	F1 CV	Umbral
AdaBoost	Sin rebalanceo	0,712	0,46
AdaBoost	Pobres x4	0,714	0,55
CART	Sin rebalanceo	0,686	0,28
CART	Upsampling	0,685	0,75
CART	Downsampling	0,677	0,74
Naive Bayes	Sin rebalanceo	0,644	0,88
Naive Bayes	Pobres x4	0,644	0,96
Random Forest (grilla CV)	Sin rebalanceo	0,715	0,34
Random Forest (grilla CV)	Pobres x4	0,710	0,52

- Rebalancear cambia el F1 en menos de 1 punto, a veces para peor.
- Lo que cambia es el umbral: elegirlo para maximizar el F1 ya corrige el desbalance.

¿Qué modelo le recomendaríamos al gobierno?

Modelo	F1 CV	Variables	¿Se puede explicar?	Información que exige
LightGBM con X_7	0,768	1.296	Poco: cientos de árboles	Encuesta completa del hogar y de cada miembro
LightGBM con X_5	0,731	43	Poco: cientos de árboles	Vivienda, composición, empleo y arriendo
Logit con X_5	0,711	43	Sí: un coeficiente por variable	La misma de X_5
CART con X_5	0,686	43	Sí: reglas que se leen en un árbol	La misma de X_5

- No necesariamente el de mayor puntaje: más F1 exige más información y modelos más difíciles de explicar.
- Con las mismas 43 variables, pasar de un solo árbol al boosting suma 4,5 puntos; ampliar la información a X_7 suma 3,7 más.
- El F1 pesa igual leakage y undercoverage. Si la agencia prefiere no dejar pobres por fuera, basta bajar el umbral, sin cambiar de modelo.
- **Propuesta:** boosting con un conjunto corto de variables fáciles de verificar, y un modelo explicable como referencia para justificar a quién se focaliza.

¡Gracias!